

Data Science

Lecture 06: Decision Trees and Ensembles

Prof. Dr. Michael Granitzer

Converted from LNKB-tex part-en-ds-chp5-decision-tree-learning.tex. Builds on Lecture 04 (hypothesis space, inductive bias, bias–variance, cross-validation) and Lecture 02 (entropy is built from the probability material). The arc runs from the most interpretable model in the course to a committee that gives that interpretability up.

Module 1

Trees as Hypotheses

Guiding Question: *A linear model hides its reasoning in 200 weights. What would a model look like that can be read aloud?*

A Tree Is a Sequence of Questions

Each inner node tests one attribute, each branch is an answer, each leaf is a class.

Example	Sky	Temperature	Humidity	Wind	EnjoySport
1	sunny	warm	normal	strong	yes
2	sunny	warm	high	strong	yes
3	rainy	cold	high	strong	no
4	sunny	warm	high	strong	yes

To classify a new day, start at the root and follow the answers to a leaf.

The tree **is** the hypothesis. Unlike a weight vector, it can be read aloud as a set of rules — which is why trees are the standard choice when a decision has to be explained.

Speaker notes

Interpretability is the selling point and should be named early. Ask what "explaining" a linear model with 200 weights would look like by comparison — this is the accountability thread from Lecture 01 returning.



Construction and Classification

Construction (*recursive*)

If the data is pure enough, make a leaf.
Otherwise: choose a **split criterion**,
decompose D into $D_1, \dots, D_s$, and recurse
on each part.

Classification (*recursive*)

If the node is a leaf, return its class.
Otherwise: follow the successor selected
by the example's attribute value.

The whole learning problem reduces to one question: **which attribute to split on?**
Everything else is bookkeeping.

The template is deliberately abstract — ID3, C4.5 and CART differ only in the split criterion and the stopping rule. Say that now so the next units read as variations, not as separate algorithms.

When to Use Them

Well suited

Attribute-value data; discrete or continuous features; missing values; a need for an explainable model; no assumption of linearity.

Poorly suited

Smooth relationships (a tree approximates a diagonal boundary with a staircase); many equally relevant features; and — as we will see — stability.

Mini-Exercise

Using the four examples above, write down **by hand** a decision tree that classifies all of them correctly.

1. Can you do it with a single test?
2. How many different correct trees can you find?
3. Which of them would you trust on a new day, and why?

Mini-Exercise — Solution

Mini-Exercise — Solution

1. Yes — splitting on Sky alone separates them: sunny → yes, rainy → no.

2. Several. Temperature (warm/cold) also separates them perfectly, as do combinations that test both.

3. Undecidable from four examples. Perfect training accuracy does not choose between competing hypotheses — this is the inductive-bias problem, and the next module gives it a criterion.

Takeaway

A decision tree is a **readable hypothesis** built by recursive splitting. The only real decision is which attribute to split on next — and the data alone does not settle it.

Speaker notes

The staircase point is worth a sketch on the board: trees split axis-parallel, so a diagonal boundary costs many splits. That limitation motivates ensembles at the end of the lecture.



Module 2

Choosing a Split

Guiding Question: *Every attribute could be split on. What makes one split better than another — and how do we measure it?*

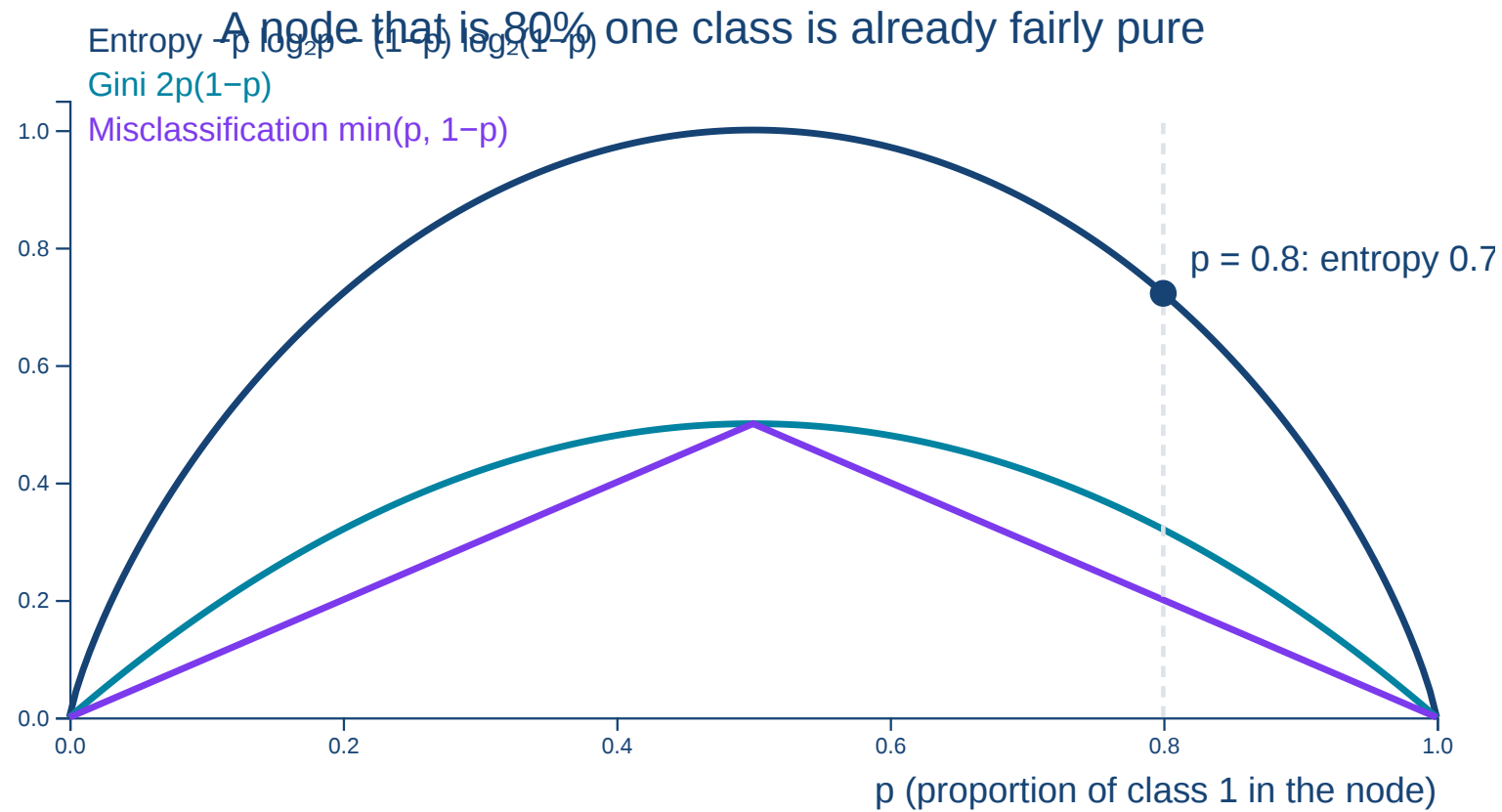
What Makes a Split Good?

A split is good if the resulting subsets are **purier** than what we started with — closer to containing a single class.

So we need a number that measures impurity: maximal when the classes are evenly mixed, zero when a node is pure.

Guiding question (P1). Let students propose a measure before showing entropy — "fraction of the majority class" is the usual proposal, and it is exactly misclassification error, whose weakness the visualization then shows.

Three Measures



Stepped viz (P6): entropy, then Gini, then misclassification, then a concrete node at $p = 0.8$. Ask before the third click how misclassification will compare — its flatness is the reason it is rarely used as a split criterion, even though it is what we ultimately care about.



Entropy

For a node with class probabilities $P(A_1), \dots, P(A_k)$:

$$H(\mathcal{A}) = - \sum_{i=1}^k P(A_i) \log_2 P(A_i)$$

measured in **bits** — the average number of yes/no questions needed to identify the class of an example in that node.

Pure node

$H = 0$ — no question needed

Two classes, 50/50

$H = 1$ bit — one question needed

k equal classes

$H = \log_2 k$ bits

Speaker notes

The "questions needed" reading makes entropy concrete and is worth more than the formula. It also makes the next slide's subtraction obviously meaningful: information gain is *questions saved*.



Conditional Entropy and Information Gain

After splitting on an attribute $\mathcal{B}$ with outcomes $B_1, \dots, B_s$:

$$H(\mathcal{A} \mid \mathcal{B}) = \sum_{j=1}^s P(B_j) \cdot H(\mathcal{A} \mid B_j)$$

— the impurity of each part, weighted by how much data lands in it.

$$\text{Gain}(\mathcal{B}) = H(\mathcal{A}) - H(\mathcal{A} \mid \mathcal{B})$$

Information gain is the impurity removed by the split. Choose the attribute that maximises it.

The weighting by $P(B_j)$ is what stops a split that isolates one example from looking good. Ask what would happen without it — a strong argument for reading formulas as decisions rather than as notation.

Mini-Exercise

A node holds 8 examples: 4 "yes", 4 "no". An attribute splits it into [3 yes, 1 no] and [1 yes, 3 no].

1. What is the entropy before the split?
2. What is the conditional entropy after it?
3. What is the information gain?

Mini-Exercise — Solution

The small number is the lesson: intuitively the split "looks decent", but it buys little. Comparing this with a perfect split ($\text{gain} = 1$) calibrates expectations.



Mini-Exercise — Solution

1. $H = 1$ bit — a perfectly even mix.

2. Each part has $H = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} \approx 0.811$; both hold half the data, so $H(\mathcal{A} \mid \mathcal{B}) \approx 0.811$.

3. Gain $\approx 1 - 0.811 = 0.189$ bits.

A split that leaves both branches 3:1 removes less than a fifth of a bit.

Takeaway

Impurity measures how mixed a node is; **information gain is the impurity a split removes**, weighted by the data it moves. Entropy and Gini agree closely; misclassification error is too flat to choose between splits.

Module 3

Building the Tree

Guiding Question: ID3 never reconsiders a split. What does that greedy choice assume about the world?

ID3

ID3(examples D , attributes A , target)

1. If all examples share one class $\rightarrow$ return a leaf with that class.
2. If A is empty $\rightarrow$ return a leaf with the majority class.
3. Otherwise choose $A^* \in A$ with the highest **information gain**.
4. For each value a of A^* : add the subtree **ID3**(D_a , $A \setminus \{A^*\}$, target).

Greedy and recursive: each split is chosen for immediate gain, and never reconsidered.

Speaker notes

Point at step 3 and name it: this is heuristic search from Lecture 04's principles unit. The hypothesis space of all trees is astronomically large; ID3 walks one path through it without backtracking.



What ID3 Assumes

Hypothesis space

All decision trees over the attributes — so the target function is certainly in there. No representational bias.

Search bias

It prefers trees that place **high-gain attributes near the root**, and therefore **shorter trees**. It stops at the first tree that fits.

Is a preference for short trees justified? It is Occam's razor as an algorithm — a genuine assumption about the world, not a mathematical necessity.

Force the interpretation (P2, P3): ask why a shorter tree should generalise better. There is an argument (fewer short trees exist, so fitting the data by accident is less likely) but it is not airtight, and students should see it as a defensible choice rather than a fact.

CART

What differs

Binary splits only **Gini** impurity instead of entropy handles **regression** by splitting on variance reduction and predicting the mean in each leaf.

What does not

The template is unchanged: choose a split, decompose, recurse. CART is a different set of choices inside the same skeleton.

Binary splits avoid a defect of multi-way splits: an attribute with many values (an ID number, in the extreme) achieves high information gain by fragmenting the data into pure singletons that generalise to nothing.

The many-valued-attribute defect is the reason C4.5 introduced gain ratio. It is a good concrete instance of a metric being gamed by the structure of the data — a theme students will meet in evaluation and in production systems.

Mini-Exercise

Your dataset includes a `customer_id` column, unique for every row.

1. What information gain does splitting on it achieve?
2. What does the resulting tree look like?
3. What does it predict for a new customer?

Mini-Exercise — Solution

The cleanest overfitting example in the course, and it arrives before pruning is introduced, which motivates the next module.

Mini-Exercise — Solution

1. Maximal — every branch holds one example, so every leaf is pure and the conditional entropy is 0.

2. A root with one branch per training example: a lookup table, one level deep.

3. Nothing useful — a new customer's id matches no branch. Perfect training purity, zero generalisation.

This is why gain must be corrected for the number of values (gain ratio), or splits restricted to binary (CART).

Takeaway

ID3 is a **greedy, non-backtracking search** whose bias is a preference for short trees. CART changes the split type and the impurity measure, not the idea. Both can be fooled by an attribute with too many values.

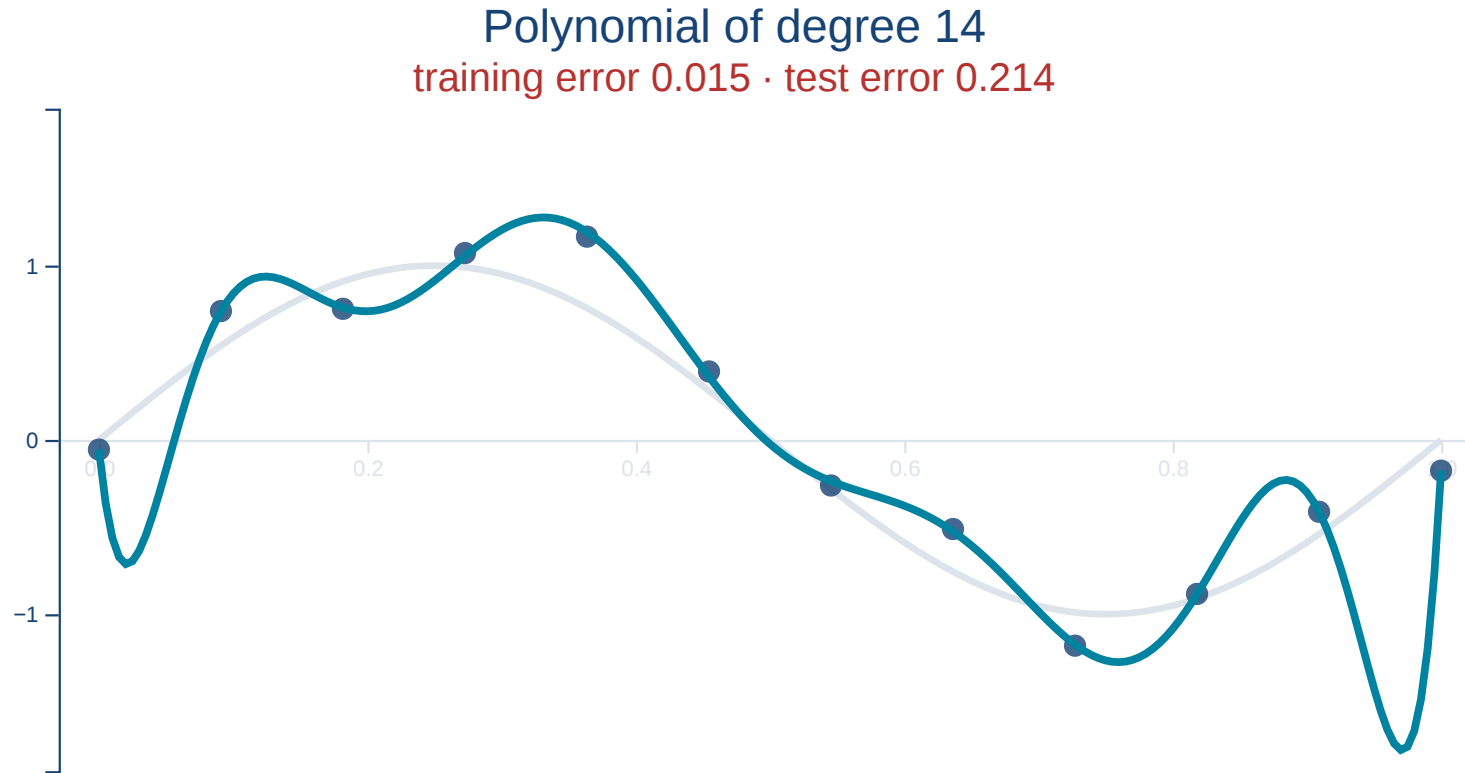
Module 4

Overfitting and Pruning

Guiding Question: *A tree can achieve 100% training accuracy on any dataset. Why is that a symptom rather than an achievement?*

A Tree Grown to Purity Is Overfitted

Left alone, ID3 splits until every leaf is pure — memorising the training set, noise included.



Tree depth plays the role of polynomial degree: more splits, lower training error, and eventually rising test error.

Deliberate reuse of the visualization from the neural-network lecture (P6 and the spiral): the same phenomenon, a third model family. Students should now recognise the shape before the caption says it.

Two Places to Intervene

Stopping (pre-pruning)

Halt splitting **during** construction — when a node is small, when gain is below a threshold, or at a depth limit.

Cheap, but short-sighted: a weak split may enable a strong one beneath it.

Pruning (post-pruning)

Grow the full tree, then **remove** subtrees that do not earn their keep.

More expensive, and generally more reliable.

The "weak split enabling a strong one" point is the standard argument for post-pruning, and it is the XOR structure again: neither attribute helps alone.

Reduced Error Pruning

1. Split the data into a **training** and a **validation** set.
2. Grow the full tree on the training set.
3. For each inner node: consider replacing its subtree with a leaf carrying the majority class.
4. Keep the replacement if accuracy on the **validation** set does not fall.
5. Repeat until no beneficial replacement remains.

The decision is made on data the tree was **not** grown on. Pruning against the training set would remove nothing — every subtree helps there by construction.

Speaker notes

This is the hold-out principle from Lecture 04, now used to make a structural decision rather than to report a number. Ask what happens if the validation set is small — pruning decisions become noisy, which is why cost-complexity pruning with cross-validation is preferred in practice.



Mini-Exercise

A tree has 100% training accuracy with 340 leaves. After pruning it has 41 leaves, 94% training accuracy and 89% validation accuracy — up from 78%.

1. What did the 299 removed leaves contain?
2. Why did training accuracy fall?
3. Was pruning worth it?

Mini-Exercise — Solution

Part 2 is where students hesitate — accepting worse training performance feels like failure. Naming it as the point of the exercise reinforces that training error was never the objective.

Mini-Exercise — Solution

1. Mostly noise — splits that separated individual training examples without capturing structure that recurs.

2. Because those examples are now misclassified. That is the intended outcome, not a regression.

3. Yes: +11 points on unseen data, and a tree small enough to read. Both axes improved.

Takeaway

An unpruned tree memorises. **Pruning trades training accuracy for generalisation**, and the trade must be judged on data the tree never saw.

Module 5

Ensembles

Guiding Question: *Change a few examples and the tree changes completely. Can that instability be turned into an advantage?*

Unstable Learners

High variance

Many parameters, so small changes in the training data produce very different models.

Runtime limits

The learner cannot search its hypothesis space exhaustively and settles for a heuristic path.

Insufficient capacity

The model family is too simple to express the target at all.

Decision trees are the textbook unstable learner: change a handful of examples and a different attribute wins at the root, producing a completely different tree.

Speaker notes

Instability is the property that makes ensembles work, so it must be understood as a fact about trees before bagging looks like anything other than a trick. Ask why a greedy root choice makes the whole tree fragile.



The Idea

Ensemble learning builds several learners and combines their predictions — by majority vote for classification, by averaging for regression.

Bagging

Bootstrap aggregating — train on resampled datasets.

Boosting

Train weak learners in sequence, each focusing on the previous one's mistakes.

Random forest

Bagging plus randomised attribute selection at every split.

Ensembles exploit instability: **many different wrong models, averaged, are right more often than any one of them.**

The takeaway is counterintuitive enough to state plainly. The condition for it to hold — errors must be at least partly independent — is what the next two units engineer deliberately.

Bootstrap Aggregating

The problem. A group of predictors helps only if they differ — but we have one algorithm and one dataset, which yield one model.

The solution. Manufacture different datasets: draw n examples from D **with replacement**, train a predictor on each sample, and combine them by vote or average.

The learner never changes. Only the data it sees does.

Bootstrapping was introduced in Lecture 04 as an error-estimation scheme here the same resampling produces model diversity. Same tool, different job — worth naming as the spiral it is.

What Resampling Actually Does

Each bootstrap sample contains about **63.2%** of the distinct examples of D . The probability that a particular example is *never* drawn in n tries is

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} e^{-1} \approx 0.368$$

The gain

Each model sees a different two-thirds of the data, so their errors are partly independent. **Averaging reduces variance** — and therefore overfitting.

The left-out third

The ~36.8% not drawn is the *out-of-bag* sample, usable as a free validation set for that model.

The 63.2% is a nice concrete consequence of a limit students can verify. The out-of-bag idea is practically important: it gives an honest error estimate without a separate hold-out.

Mini-Exercise

You bag 100 trees. Each individually achieves 70% accuracy.

1. Under what condition would the ensemble beat 70%?
2. What happens if all 100 trees are identical?
3. Does bagging reduce bias, variance, or both?

Mini-Exercise — Solution

Part 2 is the crux and motivates random forests directly: bootstrap samples alone may not decorrelate trees enough, especially when one attribute dominates.

Mini-Exercise — Solution

1. Their errors must be at least partly independent — different trees must be wrong about different examples for the vote to correct them.

2. Nothing improves: 100 copies of one model vote unanimously and reproduce its 70%.

3. Mainly **variance.** Averaging does not fix a systematic error every model shares.

Takeaway

Bagging manufactures diversity by **resampling the data**, and averages away variance. It helps exactly to the extent that the models' errors are independent — and not at all when they are not.

Why Bagging Trees Is Not Enough

Trees stay similar

If one attribute has much higher gain than the rest, it wins at the root of **every** bootstrap sample. The trees correlate, and averaging them buys little.

And it costs

Interpretability is gone — you can no longer read the model as a set of rules — while the computation of 100 near-identical trees is largely wasted.

This is the direct answer to part 2 of the previous mini-exercise. Resampling alone does not guarantee diversity the dominant attribute defeats it.

The Fix: Randomise the Split Too

Random forest (Breiman 2001)

For each of T trees:

1. Draw a bootstrap sample from D .
2. Grow a tree — but **at every node**, choose the split from a random subset $A' \subset A$ of m_A attributes rather than from all of them.
3. Do not prune.

Combine by majority vote (classification) or average (regression).

The dominant attribute is simply **absent** from most nodes' candidate sets, so other attributes get to define the tree. Diversity is engineered rather than hoped for.

Step 2 is the entire contribution, and it is one line. Typical m_A is $\sqrt{|A|}$ for classification. Step 3 is counterintuitive — individual overfitting is tolerated because averaging handles the variance.

What You Get and What You Give Up

Gained

Strong accuracy with almost no tuning out-of-bag error for free a usable attribute-importance measure.

Lost

The readable single tree. A forest is a **committee**, and no member speaks for it.

Attribute importance from a forest is not a causal statement. It reports which attributes the trees found useful for splitting — under this data, this sampling, this correlation structure.

Speaker notes

The warning matters because attribute importance is routinely over-read in practice, including in published work. It connects back to Lecture 01's correlation/causation module.



Mini-Exercise

You must choose between a single pruned decision tree (86% accuracy) and a random forest (91%) for:

1. an internal churn-prediction model,
2. an automated loan-refusal system that must be justified to applicants.

Which do you choose in each case?

Mini-Exercise — Solution

Speaker notes

Close the lecture on this (P2, and the accountability thread from L01): model selection is a decision about consequences, not only about a metric. Expect disagreement on case 2 and let it run — that is the exercise working.



Mini-Exercise — Solution

1. The **forest**: five accuracy points matter and nobody needs the model explained.

2. The **tree** — or a forest plus a documented, auditable process. A refusal that cannot be explained may be neither acceptable nor lawful.

The best model is the one that fits the **decision**, not the leaderboard.

Takeaway

Random forests **engineer diversity** by randomising the attributes available at each split, turning unstable trees into a stable committee — at the cost of the one thing a single tree was best at: being readable.